ENGIQ

Engineering Intelligence Platform

Business Architecture · System Design · Market-Ready Scaffold

Confidential · Version 1.0 · March 2026

1. Business Architecture

EngineIQ operates as a B2B SaaS platform. Understanding where it sits in both the business and technology layer is critical for go-to-market clarity, pricing decisions, and product sequencing.

1.1 Where EngineIQ Sits in the Business Layer

EngineIQ is a horizontal engineering infrastructure product. It does not belong to any single vertical industry — it serves any company that builds software. Within a client's business, it sits between the engineering function and the product delivery function.

Business Layer	Where EngineIQ Lives	Stakeholder
Executive / Board	Delivery risk visibility & engineering ROI	CTO, CEO
Engineering Management	Sprint forecasting, architecture governance	Engineering Manager, VP Eng
Team Lead / Principal	PR review, standards enforcement	Tech Lead, Senior Engineer
Developer	Code feedback, test generation	Mid / Junior Developer

KEY EngineIQ enters at the Team Lead layer (lowest friction) and expands upward to Engineering Management and CTO as the client matures. This is the land-and-expand motion.

1.2 Business Model

EngineIQ follows a tiered SaaS subscription model, billed monthly in South African Rand. Pricing is per-repository, not per-seat, which is more palatable to SA engineering managers who resist headcount-based pricing.

Tier	Starter	Growth	Scale
Price / month	R 2,500	R 5,500	R 12,000
Repositories	Up to 3	Up to 10	Unlimited
PR Review	✓	✓	✓
Architecture Linter	—	✓	✓
Delivery Forecasting	—	—	✓
Dashboard & Reports	Basic	Full	Full + API
Data Residency (SA)	✓	✓	✓ + Private Deploy
SLA	Best effort	99.5%	99.9% + CSM

1.3 Go-To-Market Motion

The GTM strategy is a two-stage land-and-expand play specific to the SA engineering ecosystem.

Stage 1 — Land (Month 1–6)

- Use Billable as the reference client. Build real metrics over 90 days.
- Target FinTech and HealthTech CTOs in JHB and CT through direct outreach.
- Lead with the POPIA / data residency angle — no competitor can match this.
- Offer a 30-day free trial on the Starter tier with no credit card required.
- Position pricing in rand — this is a structural cost advantage vs global tools.

Stage 2 — Expand (Month 6–18)

- Upgrade Starter clients to Growth tier using architecture drift reports as the trigger.
- Introduce the Sprint Risk Forecasting module to Engineering Managers.
- Use dashboard metrics from existing clients as the sales asset for new prospects.
- Partner with SA developer communities: DevConf, ZaTech Slack, Jozi JUG.

1.4 Customer Journey Map

Aware	Trial	Activate	Expand	Advocate
CTO hears about EngineIQ via ZaTech or LinkedIn	30-day free trial on Starter. First PR reviewed within 10 min.	Team sees value. CTO reviews the 30-day metrics report.	Upgrades to Growth. Architecture linter enabled.	CTO refers EngineIQ at a meetup or case study interview.

2. System Architecture

EngineIQ is a cloud-native SaaS platform built on Azure, operating exclusively in the South Africa North region (Johannesburg) to satisfy POPIA data residency requirements. All source code is processed ephemerally — never persisted to disk or database.

2.1 Architecture Principles

- Ephemeral code processing — diffs in memory only, findings persisted.
- Tenant isolation at every layer — no cross-tenant data leakage possible.
- Hybrid review engine — deterministic rules first, AI second.
- Async by default — webhooks return immediately, processing queued.
- Observable by design — structured logging and metrics from day one.
- POPIA-compliant — all data processed and stored in Azure SA North.

2.2 System Layers

The platform is divided into five distinct layers, each with clear boundaries and responsibilities.

Layer	Responsibility	Technology
1. Ingestion	Receive GitHub webhooks, validate HMAC signature, enqueue PR jobs	.NET 8 Web API, Azure Service Bus
2. Context	Index repo structure, detect architecture style, build semantic context via embeddings	Azure AI Search, Redis Cache
3. Review Engine	Run deterministic rules, apply AI analysis, score severity of findings	Azure OpenAI, .NET Background Worker
4. Feedback	Classify findings, format PR comments, post back to GitHub	GitHub REST API, .NET 8
5. Intelligence	Store findings, serve dashboard, generate trend reports, expose metrics API	PostgreSQL, .NET 8 API, React Dashboard

2.3 Component Map

Component	Function	Boundary
Webhook Listener	Receives PR events from GitHub. Validates HMAC. Enqueues job to Service Bus. Returns 200 immediately.	Public HTTPS endpoint
PR Job Processor	Dequeues PR job. Fetches diff via GitHub API. Passes to Review Orchestrator. Handles retries.	Internal worker
Context Builder	Indexes repo folder structure. Detects architecture style (Clean, Layered, Hexagonal). Caches per tenant.	Internal service

Standards Engine	Loads tenant YAML config. Runs deterministic rule checks. Produces structured violation list.	Internal service
AI Review Engine	Sends structured diff + context to Azure OpenAI. Extracts findings. Deduplicates against rule results.	Internal, no raw code sent
Feedback Generator	Merges all findings. Classifies by severity. Formats PR comment. Posts to GitHub via App token.	Outbound to GitHub
Audit & Metrics Store	Persists finding metadata (never source code). Powers dashboard. Enables trend reporting.	PostgreSQL — tenant-isolated
Tenant Management API	Handles onboarding, GitHub App installation, YAML config management, billing status.	Public API, auth required
Dashboard (React)	Visualises PR review history, violation trends, architecture drift score, team metrics.	Frontend SPA

2.4 Data Flow — PR Review Lifecycle

The following sequence describes the complete lifecycle of a single PR review event from webhook receipt to GitHub comment.

Step	Action	Component	Output
1	GitHub fires PR webhook to EngineIQ endpoint	Webhook Listener	HTTP 200 + job enqueued
2	Worker dequeues job, fetches PR diff via GitHub API	PR Job Processor	Raw diff in memory
3	Context Builder checks Redis for cached repo context. If stale, re-indexes.	Context Builder + Redis	Repo context object
4	Standards Engine loads tenant YAML, runs deterministic checks against diff	Standards Engine	Deterministic findings list
5	AI Engine receives structured diff + context (no raw secrets). Calls Azure OpenAI.	AI Review Engine	AI findings list
6	Feedback Generator merges, deduplicates, classifies severity, formats comment	Feedback Generator	Structured PR comment
7	Comment posted to GitHub PR via GitHub App installation token	GitHub REST API	Visible PR review comment
8	Finding metadata (never source code) persisted to PostgreSQL for reporting	Audit Store	Metrics available in dashboard

SEC At no point is raw source code written to disk or database. Steps 2–6 operate entirely in-memory. Only structured findings metadata is persisted. Azure OpenAI is called under a zero-training data processing agreement.

3. Infrastructure Design

All infrastructure runs in Azure South Africa North (Johannesburg) to satisfy POPIA data residency requirements. The architecture is containerised, horizontally scalable, and cost-optimised for SME-scale launch volumes.

3.1 Azure Services Map

Service	Purpose	Tier	Est. R/mo
Azure Container Apps	Hosts API, background worker, dashboard backend. Scales to zero.	Consumption	R 800–1,500
Azure Service Bus	Async PR job queue. Retry, dead-letter, idempotency.	Standard	R 200–400
Azure OpenAI	GPT-4o model endpoint. Zero-training DPA. SA region.	Standard	R 1,500–4,000
Azure AI Search	Tenant-isolated vector index for repo context embeddings.	Basic	R 800–1,500
Azure Cache for Redis	Repo context cache per tenant. TTL-controlled expiry.	C1 Basic	R 400–800
Azure Database for PostgreSQL	Finding metadata, tenant config, audit log, metrics.	Burstable B2s	R 600–1,200
Azure Key Vault	GitHub App private key, API keys, connection strings.	Standard	R 100–200
Azure Monitor + App Insights	Structured logging, telemetry, alerting, SLA tracking.	Pay-as-you-go	R 200–500
TOTAL	**Estimated monthly infrastructure cost at launch (5–20 clients)**		**R 4,600–10,100**

3.2 Security Architecture

- GitHub webhook HMAC-SHA256 signature validation on every inbound request.
- All secrets retrieved from Azure Key Vault via managed identity — no env vars with secrets.
- Tenant ID injected at Service Bus message level — workers enforce tenant boundary.
- PostgreSQL row-level security enforces tenant isolation at database layer.
- Azure OpenAI called over private endpoint — no public internet traversal for code data.
- GitHub App installation tokens scoped per repository, short-lived, never stored.
- Full audit log of every review event: timestamp, tenant, PR ID, findings count — no code.

4. Solution Scaffold — Repository Structure

The following structure represents the complete market-ready scaffold. Each project has a defined responsibility and clear dependency boundary. All projects follow Clean Architecture principles.

4.1 Repository Layout

Path	Responsibility
src/EngineIQ.API	Public HTTP layer. Webhook listener, tenant management endpoints, auth middleware.
src/EngineIQ.Worker	Background job processor. Dequeues PR jobs from Service Bus. Orchestrates review pipeline.
src/EngineIQ.ReviewEngine	Core review orchestrator. Coordinates Context Builder, Standards Engine, AI Engine, Feedback Generator.
src/EngineIQ.StandardsEngine	Deterministic rule executor. Loads YAML config. Runs boundary, naming, coupling checks.
src/EngineIQ.ContextBuilder	Repository indexer. Detects architecture style. Produces context object for AI enrichment.
src/EngineIQ.AIEngine	Azure OpenAI integration. Prompt orchestration. Response parsing. Finding extraction.
src/EngineIQ.FeedbackGenerator	Finding merger, severity classifier, PR comment formatter, GitHub comment poster.
src/EngineIQ.Domain	Shared domain entities, value objects, interfaces, enums. No external dependencies.
src/EngineIQ.Infrastructure	PostgreSQL repositories, Redis cache client, Service Bus client, Azure Key Vault client.
src/EngineIQ.GitHub	GitHub App client. Webhook validation, diff fetching, comment posting, installation token management.
src/EngineIQ.Dashboard	React SPA. PR review history, violation trends, architecture drift score, tenant settings.
config/standards-templates/	Default YAML architecture standards configs. One per detected architecture style.
tests/EngineIQ.Tests.Unit	Unit tests for Standards Engine, AI response parsing, finding classification logic.
tests/EngineIQ.Tests.Integration	Integration tests for GitHub webhook processing, PostgreSQL repositories, Service Bus flow.
infra/	Azure Bicep IaC templates. One-command environment provisioning.
docs/	Architecture decision records (ADRs), API docs, onboarding guide, DPA template.

4.2 Domain Model — Core Entities

Entity	Key Properties	Notes
Tenant	TenantId, Name, Plan, GitHubOrgId, CreatedAt	Root aggregate. All data scoped to TenantId.
Repository	RepoId, TenantId, FullName, ArchitectureStyle, LastIndexedAt	One tenant can own many repos.
PRReviewJob	JobId, RepoId, PRNumber, Status, CreatedAt, CompletedAt	Created on webhook. Status: Queued → Processing → Complete.
Finding	FindingId, JobId, Severity, Category, FilePath, LineNumber, Message, Source	Source: Rule | AI. Never contains source code.
ArchitectureRule	RuleId, TenantId, Name, Layer, Expression, Severity	Loaded from tenant YAML config at review time.
ReviewComment	CommentId, JobId, GitHubCommentId, PostedAt	Links finding batch to the GitHub PR comment.
TenantMetrics	PRsReviewed, ViolationsFound, ViolationsPrevented, AvgReviewMs	Aggregated. Used for dashboard and ROI reporting.

5. Build Milestones — Market-Ready Sequence

The following milestones represent the ordered build sequence from zero to market-ready. Each milestone produces a working, testable increment. Billable is the internal test client throughout.

Milestone 0 — Working Loop (Weeks 1–2)

GOAL A GitHub PR in the Billable repo receives a real AI-generated review comment. Nothing more, nothing less.

- Register GitHub App on Billable org.
- Scaffold EngineIQ.API with a single /webhook endpoint.
- Validate HMAC signature. Log the payload.
- Fetch the PR diff via GitHub REST API.
- Send diff to Azure OpenAI. Parse response.
- Post comment back to the PR.
- Deploy to Azure Container Apps (SA North).

Deliverable: First automated PR comment on Billable. The loop is real.

Milestone 1 — Standards Engine (Weeks 3–5)

GOAL Deterministic architecture rules run before AI, reducing false positives and cost.

- Build EngineIQ.Domain — entities, interfaces, enums.
- Build EngineIQ.StandardsEngine — YAML loader, rule executor.
- Create default clean-architecture.yaml standards template.
- Integrate standards results into PR comment (severity-classified).
- Wire Service Bus — webhook enqueues, worker processes.
- Add PostgreSQL — tenant and finding persistence.

Deliverable: Architecture rule violations appear in Billable PRs, separated from AI findings.

Milestone 2 — Context Builder (Weeks 6–8)

GOAL Reviews become architecture-aware, not just diff-aware.

- Build EngineIQ.ContextBuilder — folder scanner, layer detector.
- Integrate Azure AI Search — repo embeddings per tenant.
- Add Redis — cache repo context with TTL.
- Enrich AI prompts with repo context (layer map, detected patterns).
- Add architecture style detection (Clean, Layered, Hexagonal, Modular Monolith).

Deliverable: AI review comments reference the actual architecture of Billable's codebase.

Milestone 3 — Multi-Tenancy (Weeks 9–12)

GOAL The platform can onboard and isolate multiple clients safely.

- Build Tenant Management API — registration, GitHub App install flow, config management.
- Enforce tenant isolation: Service Bus, Redis, AI Search, PostgreSQL RLS.
- Per-tenant YAML standards configuration via dashboard or API.
- GitHub App installation token scoping per tenant.
- Onboard first external pilot client alongside Billable.

Deliverable: Two isolated tenants running in production simultaneously without data leakage.

Milestone 4 — Dashboard & Metrics (Weeks 13–16)

GOAL CTOs have visibility into engineering health. EngineIQ becomes sticky.

- Build EngineIQ.Dashboard (React SPA).
- PR review history timeline.
- Violation trend chart (by severity, by file, by rule).
- Architecture drift score over time.
- 30-day metrics email report (auto-generated).
- Developer satisfaction feedback widget.

Deliverable: Billable CTO can present engineering health metrics to stakeholders. Sales asset created.

Milestone 5 — Production Hardening (Weeks 17–20)

GOAL The platform is sellable, observable, and defensible.

- Structured logging and Application Insights telemetry throughout.
- Retry logic and dead-letter handling on all queued jobs.
- Rate limiting on webhook endpoint.
- Large PR chunking (PRs > 2,000 lines split intelligently).
- False positive reduction — developer override mechanism.
- POPIA data processing documentation finalised.
- DPA template reviewed and signed-off.
- Azure Bicep IaC — one-command environment provisioning.
- Load testing at 10× expected volume.
- Security penetration test on public endpoints.

Deliverable: Platform is ready to sell. First paying client can be onboarded with confidence.

6. Success Metrics

EngineIQ is only credible if it is measurable. The following metrics are tracked from day one across two categories: platform performance and client value delivered.

6.1 Platform Metrics

Metric	Target	Why It Matters
PR review completion time	< 90 seconds	Developer experience threshold
Webhook → comment latency	< 120 seconds	Includes queue + processing time
False positive rate	< 15%	Trust threshold — above this, devs ignore the tool
Suggestion acceptance rate	> 40%	Indicator of finding quality
Platform uptime	> 99.5%	SLA baseline for Growth tier
Token cost per PR review	< R 8	Unit economics must stay positive at R2,500/mo

6.2 Client Value Metrics

Metric	Target (90 days)	Sales Use
PR cycle time reduction	> 30%	Headline ROI metric for CTOs
Architecture violations caught pre-merge	Track absolute number	Quantifies defect prevention value
Change failure rate trend	Downward trend	Links EngineIQ to production stability
Senior reviewer time saved	> 4 hrs/week	Translate to rand value for ROI case
Developer NPS on tool	> 30	Adoption signal — below this, churn risk

7. Default Standards Configuration

Every tenant receives a default YAML standards file on onboarding, pre-populated based on the detected architecture style. The following is the default for Clean Architecture (.NET).

```yaml
# EngineIQ Standards Configuration — Clean Architecture (.NET)
# Version: 1.0  |  Auto-generated by EngineIQ on onboarding

architecture:
  style: clean-architecture
  layers:
    - name: Domain      folders: [Domain, Core]
    - name: Application  folders: [Application, UseCases]
    - name: Infrastructure  folders: [Infrastructure, Persistence]
    - name: API          folders: [API, Controllers, WebAPI]

rules:
  - id: ARCH-001
    name: No domain dependency on infrastructure
    severity: critical
    check: layer_dependency  from: Domain  disallows: [Infrastructure, API]
  - id: ARCH-002
    name: No business logic in controllers
    severity: high
    check: layer_logic  layer: API  disallow_patterns: [if, switch, calculation]
  - id: SEC-001
    name: No hardcoded secrets or connection strings
    severity: critical
    check: secret_pattern
  - id: PERF-001
    name: No synchronous database calls in async context
    severity: high
    check: async_pattern  disallow: [.Result, .Wait(), GetAwaiter().GetResult()]
```

8. Immediate Next Steps

The following actions should be completed in sequence to begin Milestone 0. Each action has a clear owner and output.

#	Action	Output	When
1	Register GitHub App on Billable GitHub org with PR read/write permissions	App ID + Private Key	Day 1
2	Provision Azure resource group in South Africa North via Bicep	Azure environment live	Day 1
3	Scaffold EngineIQ solution with all project stubs and dependency references	Compiling solution	Day 2
4	Implement Webhook Listener with HMAC validation and Service Bus enqueue	Webhook receives events	Day 3
5	Implement PR diff fetch via GitHub REST API	Diff in memory	Day 4
6	Wire Azure OpenAI call with basic prompt and response parsing	AI findings extracted	Day 5
7	Post formatted comment back to GitHub PR via GitHub App token	First live PR comment	Day 6
8	Deploy to Azure Container Apps and test against a real Billable PR	Milestone 0 complete	Day 7–10

START Action 1 can be completed in 15 minutes. Register the GitHub App at github.com/settings/apps/new. The Private Key and App ID are the only credentials needed to begin.